ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку ИИ-ассистента

Умный Telegram-бот отдела продаж стройматериалов с автоматическим расчетом цен и логистики

Версия:	2.0 (As-Built / Фактическая реализация)
Дата:	16.10.2026
Автор:	Филимонов Тимофей
Статус:	Реализован и протестирован
Стек:	n8n + PostgreSQL + GigaChat + Telegram API

СОДЕРЖАНИЕ

1. Общая информация о проекте

2. Цели и задачи

3. Пользователи и сценарии

4. Функциональные требования

5. Данные и база знаний

6. Технические требования

7. Ограничения и риски

8. План реализации

9. Критерии приемки

БЛОК 1. Системный промпт для GigaChat

БЛОК 2. Архитектура данных в PostgreSQL

БЛОК 3. Архитектура Workflow в n8n

БЛОК 4. Маскирование ПДн (152-ФЗ)

БЛОК 5. Примеры кода ключевых нод

БЛОК 6. Тестирование и результаты

Приложения

1. Общая информация о проекте

1.1 Описание компании/контекста

Сфера деятельности — Торгово-дистрибьюторская компания. Продажа оборудования и комплектующих (около 500 уникальных SKU). Клиентская база — Смешанная: розничные покупатели (физлица) и оптовые B2B-клиенты.

Текущие процессы: Запросы на цены, наличие и доставку поступают через Telegram. Менеджеры обрабатывают их вручную: проверяют прайс, рассчитывают логистику.

Инфраструктура: Проект развернут на сервере в Нидерландах (VPS Timeweb Cloud) для обеспечения стабильной работы вебхуков Telegram и n8n.

1.2 Проблема, которую решаем

Менеджеры тратят до 40% рабочего времени на рутинные операции: проверку цен, уточнение остатков и ручной расчет доставки. Это приводит к задержкам ответов (до 12 часов) и потере «горячих» лидов.

2. Цели и задачи

2.1 Главная цель проекта

Создать Telegram-бота, который будет автоматически обрабатывать запросы клиентов по 500 SKU, автоматически определять тип клиента (розница/опт) по Telegram ID, консультировать по товарам, рассчитывать стоимость доставки на основе жестких SQL-правил и формировать текстовые ответы — чтобы сократить время ответа до 15 секунд.

2.2 Измеримые показатели успеха (KPI)

Метрика	Целевое значение	Как измеряется
Скорость первичного ответа	До 15 секунд	Время между сообщением и ответом бота
Точность распознавания товара	≥ 90%	Корректный поиск через FTS/Trigram в БД
Точность расчетов	100%	Цены и доставка берутся из SQL, LLM не считает
Полнота логирования	100% диалогов	Запись в таблицу dialogue_logs с учетом токенов

3. Пользователи и сценарии

3.1 Целевые пользователи

- **Группа 1: Розничные клиенты.**

Физлица. Пишут в Telegram. Ожидают мгновенный ответ с розничной ценой.

- **Группа 2: Оптовые клиенты.**

Зарегистрированные дистрибьюторы. Их Telegram chat_id уже внесен в базу данных. Ожидают оптовую цену и скидку.

3.2 Пользовательские сценарии (Use Cases)

Сценарий 1. Быстрая проверка цены и наличия

- Триггер: Сообщение: «Сколько стоит перфоратор Bosch и сколько их на складе?»
- Действие бота: 1. Определяет chat_id. 2. Ищет товар в БД через полнотекстовый поиск. 3. Подставляет цену (розница/опт) в промпт.
- Результат: Ответ за 10 сек с точной ценой и остатком.

Сценарий 2. Расчет заказа с доставкой

- Триггер: Сообщение: «Нужно 50 болгарок Makita с доставкой в Бишкек».
- Действие бота: 1. Находит товар. 2. Через SQL-джойн находит зону доставки для города «Бишкек» и ее стоимость. 3. Передает все цифры в GigaChat.
- Результат: Сформированный текст с итоговой суммой и сроками.

4. Функциональные требования

4.1 Основные функции (Must Have)

1. Приём запросов: Только через Telegram Bot API.
2. Автоматическая идентификация клиента: Проверка chat_id в таблице clients. Если клиент найден — применяется его тип (опт/розница) и процент скидки.
3. Умный поиск товаров: Использование PostgreSQL Full-Text Search (FTS) и Trigram similarity для распознавания названия товара из свободного текста пользователя.
4. Расчет доставки через БД: Поиск города в таблице синонимов (city_synonyms) и джойн с таблицей правил доставки (delivery_rules) для получения точной стоимости и сроков.
5. Генерация ответа через LLM: Формирование финального текста через GigaChat на основе строго переданных цифр из БД.
6. Логирование: Запись каждого диалога в таблицу dialogue_logs (текст клиента, ответ бота, использованные токены, тип клиента).

4.2 Желательные функции (Should Have)

- Анализ «слепых зон» через выгрузку из dialogue_logs.
- Различение тона ответа (краткий для Telegram).

4.3 Исключения (Won't Have)

- Канал Email (не реализован).
- Сложная авторизация по ИНН/Email в реальном времени (не реализована, используется только chat_id).

- Векторный поиск (RAG/pgvector) для регламентов (не реализован, доставка ищется через SQL).
- Эскалация на менеджера отдельной нодой (не реализована, fallback обрабатывается самим LLM).
- Генерация PDF, интеграция с 1С/CRM, голосовые сообщения.

5. Данные и база знаний

5.1 Источники данных

Источник	Формат	Метод поиска
Прайс-лист и остатки	PostgreSQL (products)	FTS (tsvector) + Trigram (pg_trgm)
База клиентов и скидки	PostgreSQL (clients)	Точное совпадение по chat_id
Правила доставки и города	PostgreSQL (delivery_rules, city_synonyms)	SQL JOIN по названию города

5.2 Требования к данным

- Конфиденциальность: Оптовые цены выдаются только если chat_id клиента есть в таблице clients с типом wholesale.
- Целостность: LLM запрещено вычислять математику. В промпт передаются только готовые числа из SQL-запросов.

6. Технические требования

6.1 Платформа и интеграции

Компонент	Выбранный инструмент	Обоснование / Фактическая реализация
Интерфейс	Telegram Bot API	Единственный канал связи. Быстрый запуск через Webhook.
LLM провайдер	GigaChat	Российская модель, 152-ФЗ. В коде используется модель "GigaChat".
Оркестратор	n8n (self-hosted)	Визуальное построение пайплайна. Развернут на VPS в Нидерландах.
Хранилище	PostgreSQL	Единая БД. Используются расширения pg_trgm (для нечеткого поиска товаров) и стандартный FTS. Векторная БД (pgvector) не используется.
Инфраструктура	VPS Timeweb Cloud (Нидерланды)	Сервер обрабатывает вебхуки Telegram и n8n для стабильности.

6.2 Нефункциональные требования

- Время ответа: до 15 секунд.
- Безопасность: API-ключи GigaChat и Telegram хранятся в credentials n8n. Данные клиентов в РФ.

7. Ограничения и риски

7.1 Ограничения проекта

- Бюджет: до 3 000 ₽/мес на API GigaChat.
- Сроки: 2 недели до запуска MVP.

7.2 Риски и митигация

Риск	Митигация (реализовано в коде)
Опечатки в названии товара	Использование Trigram similarity (similarity > 0.3) в SQL-запросе.
Галлюцинации LLM в цифрах	Математика исключена. В промпт вставляются только жесткие значения из SQL (price, base_cost).
Нераспознанный город	SQL возвращает NULL, LLM по системному промпту отвечает: «Регион не найден, уточните город».

8. План реализации

Этап	Задачи	Результат
1. Подготовка БД	Создание таблиц products, clients, delivery_rules, city_synonyms. Настройка индексов FTS и Trigram.	Готовая PostgreSQL БД
2. Сборка n8n	Настройка Telegram Trigger, SQL-нод, HTTP Request к GigaChat OAuth и Chat API.	Рабочий пайплайн
3. Промпт-инжиниринг	Написание системного промпта, который запрещает считать и требует использовать только переданные JSON-данные.	Стабильные ответы
4. Деплой	Настройка вебхуков на сервере в Нидерландах, подключение к БД.	Релиз MVP

9. Критерии приемки

- ☑ Бот корректно находит товар даже при опечатках (благодаря Trigram).
- ☑ Оптовая цена показывается только тем chat_id, которые есть в таблице clients.

- Стоимость доставки рассчитывается строго по таблице delivery_rules, LLM не выдумывает тарифы.
- ☑ Все диалоги пишутся в таблицу dialogue_logs с подсчетом tokens_used.
- Система работает стабильно на VPS в Нидерландах.

БЛОК 1. СИСТЕМНЫЙ ПРОМПТ ДЛЯ GigaChat (Фактический)

Ниже приведен полный текст системного промпта, который передается в GigaChat при каждом запросе. Промпт содержит жесткие ограничения для предотвращения галлюцинаций и соблюдения 152-ФЗ.

```
Ты — ИИ-ассистент отдела продаж компании стройматериалов.
ЖЁСТКИЕ ПРАВИЛА (152-ФЗ и точность данных):
1. ТЫ НЕ СЧИТАЕШЬ МАТЕМАТИКУ. Используй ТОЛЬКО предоставленные ниже цифры из БД.
2. НИКОГДА не выдумывай цены, наличие или условия доставки. Если в данных выше
   указано "Товар не распознан" или "Регион не найден" — честно сообщи об этом клиенту.
3. НИКОГДА не повторяй ИНН, паспорта, email и телефоны.
4. Если клиент НЕ авторизован (тип: retail), а просит опт, ответь: "Оптовые цены
   доступны только авторизованным дистрибьюторам."

ДАННЫЕ ИЗ БД (Используй ТОЛЬКО их):
- Клиент: {firstName} (Тип: {clientType}, Скидка: {discount}%)
- Товар: {productContext}
- Доставка: {deliveryContext}

ВОПРОС КЛИЕНТА: "{userText}"

ТВОЯ ЗАДАЧА:
Сформируй краткий, вежливый и деловой ответ (2-4 предложения). Назови точную цену,
остаток, стоимость и сроки доставки, если они есть в данных выше. В конце предложи
оформить заказ.
```

[SYSTEM PROMPT]

БЛОК 2. АРХИТЕКТУРА ДАННЫХ В POSTGRESQL (Фактическая)

Ниже приведены SQL-скрипты создания всех таблиц, используемых в проекте. База данных развернута на PostgreSQL с расширениями pg_trgm для нечеткого поиска.

```
-- Таблица клиентов (идентификация по chat_id)
CREATE TABLE clients (
    chat_id BIGINT PRIMARY KEY,
    client_type VARCHAR(20), -- 'retail' или 'wholesale'
    discount_percent INT,
    first_name VARCHAR(100)
);

-- Таблица товаров (с поддержкой нечеткого поиска)
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    price_retail DECIMAL,
    price_wholesale DECIMAL,
    stock_quantity INT,
    unit VARCHAR(20)
);
-- Индексы для FTS и Trigram создаются отдельно

-- Справочник синонимов городов и правил доставки
CREATE TABLE city_synonyms (
    city_name VARCHAR(100),
    synonym VARCHAR(100)
);

CREATE TABLE delivery_rules (
    zone VARCHAR(100),
    base_cost DECIMAL,
    delivery_days VARCHAR(50),
    rule_text TEXT
);

-- Логи диалогов
CREATE TABLE dialogue_logs (
    id SERIAL PRIMARY KEY,
    chat_id BIGINT,
    client_type VARCHAR(20),
    user_message TEXT,
    bot_response TEXT,
    tokens_used INT,
    channel VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

[SQL]

Пример SQL-запроса для поиска товара (FTS + Trigram)

```
WITH search_words AS (
  SELECT regexp_replace(word, '[^a-яёa-z0-9]', '', 'g') AS clean_word
  FROM unnest(string_to_array(lower('{{ $json.user_message }}'), ' ')) AS word
  WHERE length(regexp_replace(word, '[^a-яёa-z0-9]', '', 'g')) > 3
),
fts_matches AS (
  SELECT name, price_retail, price_wholesale, stock_quantity, unit,
         ts_rank(to_tsvector('russian', name), to_tsquery('russian', clean_word)) AS
rank
  FROM search_words, products
  WHERE to_tsvector('russian', name) @@ to_tsquery('russian', clean_word)
),
trigram_matches AS (
  SELECT name, price_retail, price_wholesale, stock_quantity, unit,
         similarity(name, clean_word) AS sim
  FROM search_words, products
  WHERE similarity(name, clean_word) > 0.3
)
SELECT name, price_retail, price_wholesale, stock_quantity, unit
FROM fts_matches
UNION
SELECT name, price_retail, price_wholesale, stock_quantity, unit
FROM trigram_matches
ORDER BY name
LIMIT 1;
```

[SQL]

БЛОК 3. АРХИТЕКТУРА WORKFLOW В n8n (Фактическая)

Ниже приведена схема пайплайна, реализованного в n8n. Все ноды соединены в единую цепочку обработки запроса.

```
Telegram Trigger
    ↓
Code in JavaScript ParseTelegramMessage (извлечение chat_id, username, user_message)
    ↓
┌──────────────────────┬──────────────────────┬──────────────────────────────┐
↓                      ↓                      ↓                              │
SQL Клиент             SQL Product search     SQL_Delivery1                  │
(проверка chat_id)     (FTS + trigram)        (city_synonyms + rules)       │
└──────────────────────┴──────────────────────┴──────────────────────────────┘
    ↓
Merge (сборка 3-х потоков данных)
    ↓
Code in JavaScript BuildPromptAndMaskPII (формирование промпта + маскирование ПДн)
    ↓
HTTP Request GigaChat token (OAuth 2.0, получение access_token)
    ↓
HTTP Request GigaChat API (отправка промпта, получение ответа + tokens_used)
    ↓
HTTP Request Send Telegram Message (доставка ответа клиенту)
    ↓
Insert rows in dialogue_logs (логирование в PostgreSQL)
```

[WORKFLOW SCHEMA]

Описание ключевых нод

№	Название ноды	Тип	Назначение
1	Telegram Trigger	Trigger	Точка входа. Ловит входящие сообщения от пользователей 24/7.
2	ParseTelegramMessage	Code (JavaScript)	Парсер. Извлекает chat_id, username и текст запроса.
3	SQL Клиент	PostgreSQL	Авторизация. Проверяет chat_id в БД. Возвращает тип клиента и скидку.
4	SQL Product search	PostgreSQL	Умный поиск. Ищет товар через FTS + триграммы.
5	SQL_Delivery1	PostgreSQL	Логистика. Ищет город в city_synonyms, возвращает правила доставки.

6	Merge	Merge	Сборщик. Склеивает результаты 3-х SQL-запросов в один поток.
7	BuildPromptAndMaskPII	Code (JavaScript)	Мозг системы. Формирует промпт и маскирует ПДн.
8	GigaChat token	HTTP Request	Ключ доступа. Получает access_token по OAuth 2.0.
9	GigaChat API	HTTP Request	Генерация ответа. Отправляет промпт, получает ответ + токены.
10	Send Telegram Message	HTTP Request	Отправка. Доставляет ответ пользователю в Telegram.
11	Insert rows in dialogue_logs	PostgreSQL	Аудит. Записывает историю диалога в БД.

БЛОК 4. МАСКИРОВАНИЕ ПДн (Фактическая реализация 152-ФЗ)

В соответствии с требованиями 152-ФЗ, персональные данные клиентов (chat_id Telegram) маскируются перед записью в таблицу логов dialogue_logs. Полный chat_id используется только для отправки ответа через Telegram API.

Алгоритм маскирования

```
// Маскирование chat_id для логирования (152-ФЗ)
const chatIdStr = String(chatId);
const maskedChatId = chatIdStr.length > 6
  ? chatIdStr.substring(0, 3) + '****' + chatIdStr.substring(chatIdStr.length - 3)
  : '***';

// Пример:
// Было: 174236684
// Стало: 174****684
```

[JAVASCRIPT]

Таблица видимости данных

Данные	Полный вид	Маскированный вид	Кто видит
chat_id	174236684	—	Только Telegram API (для отправки ответа)
log_chat_id	Не хранится	174****684	Администратор системы (в БД и логах)
user_message	Полный текст	Обрезан до 50 символов	Администратор (в БД)
system_prompt	Содержит имя клиента	Без ИНН/телефона/email	GigaChat API (но без ПДн)

БЛОК 5. ПРИМЕРЫ КОДА КЛЮЧЕВЫХ НОД

5.1 Нода ParseTelegramMessage

```
const tgData = $('Telegram Trigger').first().json;

return [{
  json: {
    chat_id: tgData.message?.chat?.id || tgData.chat?.id || tgData.chat_id,
    username: tgData.message?.from?.username || tgData.from?.username || '',
    user_message: tgData.message?.text || tgData.text || ''
  }
}];
```

[JAVASCRIPT]

5.2 Нода BuildPromptAndMaskPII (фрагмент)

```
// 1. Получаем данные из SQL-нод и Telegram
const tgData = $('Telegram Trigger').first().json;
const clientData = $('Execute a SQL query Клиент').first()?.json || {};
const productData = $('Execute a SQL query Product search').first()?.json || {};
const deliveryData = $('SQL_Delivery1').first()?.json || {};

// 2. Базовые переменные
const userText = tgData.text || tgData.user_message || 'Приветствие';
const chatId = tgData.chat_id || tgData.message?.chat?.id;

// 3. Данные клиента
const clientType = clientData?.client_type || 'retail';
const discount = clientData?.discount_percent || 0;
const firstName = clientData?.first_name || 'Клиент';

// 4. Данные товара (Fallback логика)
let productContext = "Товар не распознан. Уточни, какой товар тебя интересует.";
let isProductFound = false;

if (productData && productData.name) {
  isProductFound = true;
  const price = clientType === 'wholesale' ? productData.price_wholesale :
productData.price_retail;
  productContext = `
- Товар: ${productData.name}
- Цена для клиента: ${price} руб. за ${productData.unit}
- Остаток на складе: ${productData.stock_quantity} ${productData.unit}`;
}

// 5. Данные доставки
let deliveryContext = "Регион доставки не указан или не найден в базе.";
```

```
if (deliveryData && deliveryData.zone) {
  deliveryContext = `
- Зона доставки: ${deliveryData.zone}
- Стоимость доставки: ${deliveryData.base_cost} руб.
- Сроки: ${deliveryData.delivery_days}
- Правила: ${deliveryData.rule_text}`;
}

// 6. Системный промпт
const systemPrompt = `Ты — ИИ-ассистент отдела продаж компании стройматериалов.
ЖЁСТКИЕ ПРАВИЛА (152-ФЗ и точность данных):
1. ТЫ НЕ СЧИТАЕШЬ МАТЕМАТИКУ. Используй ТОЛЬКО предоставленные ниже цифры.
2. НИКОГДА не выдумывай цены, наличие или условия доставки.
3. НИКОГДА не повторяй ИНН, паспорта, email и телефоны.
4. Если клиент НЕ авторизован (тип: retail), а просит опт, ответь:
   "Оптовые цены доступны только авторизованным дистрибьюторам."

ДАННЫЕ ИЗ БД (Используй ТОЛЬКО их):
- Клиент: ${firstName} (Тип: ${clientType}, Скидка: ${discount}%)
${productContext}
${deliveryContext}

ВОПРОС КЛИЕНТА: "${userText}"

ТВОЯ ЗАДАЧА:
${isProductFound
  ? 'Сформируй краткий, вежливый и деловой ответ (2-4 предложения). Назови точную цену,
остаток, стоимость и сроки доставки, если они есть в данных выше. В конце предложи
оформить заказ.'
  : 'Товар не найден в базе или запрос неясен. Вежливо извинись, скажи, что не смог
найти такой товар в текущем каталоге, и попроси клиента уточнить название или задать
другой вопрос по стройматериалам.'}`;

// 7. Маскирование ПДн для логирования (152-ФЗ)
const chatIdStr = String(chatId);
const maskedChatId = chatIdStr.length > 6
  ? chatIdStr.substring(0, 3) + '****' + chatIdStr.substring(chatIdStr.length - 3)
  : '***';

// 8. Возврат данных
return {
  json: {
    chat_id: chatId,
    client_type: clientType,
    system_prompt: systemPrompt,
    user_message: userText,
    log_chat_id: maskedChatId,
    log_user_message: userText.length > 50 ? userText.substring(0, 50) + '...' :
userText,
    log_client_type: clientType
  }
};
```

[JAVASCRIPT]

БЛОК 6. ТЕСТИРОВАНИЕ И РЕЗУЛЬТАТЫ

6.1 Пример тестового запроса

Входящее сообщение от клиента (Telegram):

```
Сколько стоит кирпич с доставкой в Бишкек?
```

[USER MESSAGE]

Ответ бота:

```
Здравствуйте, TIMOFEY! Рад Вас видеть в нашем каталоге.

Цена на кирпич облицовочный красный для Вас со скидкой 15% составляет 21.68 руб.
за штуку, остаток на складе - 5000 штук. Стоимость доставки в Казахстан – 6500.00 руб.,
срок поставки 7-12 дней.

Предлагаю оформить заказ сегодня!
```

[BOT RESPONSE]

6.2 Метрики запроса

Параметр	Значение
Время ответа	10-15 секунд
Модель GigaChat	GigaChat:2.0.28.2
prompt_tokens	365
completion_tokens	87
total_tokens	452
Стоимость запроса	~0.14 руб. (по тарифу GigaChat)

6.3 Запись в таблице dialogue_logs

Поле	Значение
id	10
chat_id (маскированный)	174****684
client_type	wholesale
user_message	Сколько стоит кирпич с доставкой в Бишкек...
bot_response	Здравствуйте, TIMOFEY! Рад Вас видеть...
tokens_used	452
channel	telegram
created_at	2026-10-18T06:12:49.664Z

СООТВЕТСТВИЕ ТРЕБОВАНИЯМ ТЕХНИЧЕСКОГО ЗАДАНИЯ

1. Точность расчётов и минимизация контекста (ТЗ, п. 2.2 и Блок 1 системного промпта)

Языковая модель получает только извлечённые сущности (название товара, город, тип клиента) без передачи полных персональных данных. Математические расчёты (цена, количество, логистика) выполняются исключительно на уровне SQL-запросов и JavaScript в n8n. LLM категорически запрещено проводить вычисления или выдумывать цены, что гарантирует 100% точность расчётов и исключает галлюцинации.

2. Безопасность и соответствие 152-ФЗ (ТЗ, Блок 2, п. 1 и Блок 3, п. 3)

Реальный chat_id (который согласно ТЗ является персональными данными) используется строго для отправки ответа через Telegram API. Перед записью в базу данных (dialogue_logs) идентификатор подвергается обязательному алгоритмическому маскированию (например, 174****684). Это исключает хранение открытых ПДн в логах и обеспечивает полное соответствие требованиям законодательства РФ о локальном хранении и обезличивании данных.

3. Контроль затрат и аналитика (ТЗ, п. 4.2 и п. 7.1)

Нода запроса к GigaChat API возвращает системную метрику usage.total_tokens. Это значение напрямую сохраняется в таблицу dialogue_logs наряду с текстом запроса и ответа. Это позволяет в любой момент выполнить SQL-агрегацию для точного расчёта ежемесячных расходов на API и удержания в рамках утверждённого бюджета (до 3 000 ₽/мес).

4. Отказоустойчивость и Fallback-сценарии (ТЗ, п. 4.1)

Архитектура пайплайна предусматривает безопасную обработку отсутствующих данных. Использование опциональной цепочки (?.) и операторов по умолчанию (|| {}) в JavaScript, а также оператора UNION в SQL-запросах гарантирует, что система не прервёт выполнение с ошибкой, если товар или город не найдены в БД. В этом случае автоматически срабатывает Fallback-сценарий: бот честно сообщает об отсутствии товара в каталоге и вежливо просит уточнить запрос, предотвращая генерацию некорректных ответов.

СОГЛАСОВАНИЕ

Роль	ФИО	Подпись / Дата
Заказчик		
Исполнитель	Филимонов Т.Ю.	16.10.2026
Согласующий		